

5.5.3 goto 语句

goto 语句 (goto statement) 的作用是从 goto 语句无条件跳转到同一函数内的另一条语句。



不要在程序中使用 goto 语句，因为它使得程序既难理解又难修改。

goto 语句的语法形式是

```
goto label;
```

其中，*label* 是用于标识一条语句的标示符。带标签语句 (labeled statement) 是一种特殊的语句，在它之前有一个标示符以及一个冒号：

```
end: return; // 带标签语句，可以作为 goto 的目标
```

标签标示符独立于变量或其他标示符的名字，因此，标签标示符可以和程序中其他实体的标示符使用同一个名字而不会相互干扰。goto 语句和控制权转向的那条带标签的语句必须位于同一个函数之内。

和 switch 语句类似，goto 语句也不能将程序的控制权从变量的作用域之外转移到作用域之内：

```
// ...
goto end;
int ix = 10; // 错误：goto 语句绕过了一个带初始化的变量定义
end:
// 错误：此处的代码需要使用 ix，但是 goto 语句绕过了它的声明
ix = 42;
```

向后跳过一个已经执行的定义是合法的。跳回到变量定义之前意味着系统将销毁该变量，然后重新创建它：

```
// 向后跳过一个带初始化的变量定义是合法的
begin:
    int sz = get_size();
    if (sz <= 0) {
        goto begin;
    }
```

在上面的代码中，goto 语句执行后将销毁 *sz*。因为跳回到 *begin* 的动作跨过了 *sz* 的定义语句，所以 *sz* 将重新定义并初始化。

193

5.5.3 节练习

练习 5.22：本节的最后一个例子跳回到 *begin*，其实使用循环能更好地完成该任务。重写这段代码，注意不再使用 goto 语句。

5.6 try 语句块和异常处理

异常是指存在于运行时的反常行为，这些行为超出了函数正常功能的范围。典型的异常包括失去数据库连接以及遇到意外输入等。处理反常行为可能是设计所有系统最难的一部分。

当程序的某部分检测到一个它无法处理的问题时，需要用到异常处理。此时，检测出问题的部分应该发出某种信号以表明程序遇到了故障，无法继续下去了，而且信号的发出方无须知道故障将在何处得到解决。一旦发出异常信号，检测出问题的部分也就完成了任务。

如果程序中含有可能引发异常的代码，那么通常也会有专门的代码处理问题。例如，如果程序的问题是输入无效，则异常处理部分可能会要求用户重新输入正确的数据；如果丢失了数据库连接，会发出报警信息。

异常处理机制为程序中异常检测和异常处理这两部分的协作提供支持。在 C++ 语言中，异常处理包括：

- **throw 表达式** (throw expression)，异常检测部分使用 throw 表达式来表示它遇到了无法处理的问题。我们说 throw 引发 (raise) 了异常。
- **try 语句块** (try block)，异常处理部分使用 try 语句块处理异常。try 语句块以关键字 try 开始，并以一个或多个 **catch 子句** (catch clause) 结束。try 语句块中代码抛出的异常通常会被某个 catch 子句处理。因为 catch 子句“处理”异常，所以它们也被称作**异常处理代码** (exception handler)。
- 一套**异常类** (exception class)，用于在 throw 表达式和相关的 catch 子句之间传递异常的具体信息。

在本节的剩余部分，我们将分别介绍异常处理的这三个组成部分。在 18.1 节（第 684 页）还将介绍更多关于异常的知识。

5.6.1 throw 表达式

程序的异常检测部分使用 throw 表达式引发一个异常。throw 表达式包含关键字 throw 和紧随其后的一个表达式，其中表达式的类型就是抛出的异常类型。throw 表达式后面通常紧跟一个分号，从而构成一条表达式语句。

举个简单的例子，回忆 1.5.2 节（第 20 页）把两个 Sales_item 对象相加的程序。这个程序检查它读入的记录是否是关于同一种书籍的，如果不是，输出一条信息然后退出。

◀ 194

```
Sales_item item1, item2;
cin >> item1 >> item2;
// 首先检查 item1 和 item2 是否表示同一种书籍
if (item1.isbn() == item2.isbn()) {
    cout << item1 + item2 << endl;
    return 0; // 表示成功
} else {
    cerr << "Data must refer to same ISBN" << endl;
    return -1; // 表示失败
}
```

在真实的程序中，应该把对象相加的代码和用户交互的代码分离开来。此例中，我们改写程序使得检查完成后不再直接输出一条信息，而是抛出一个异常：

```
// 首先检查两条数据是否是关于同一种书籍的
if (item1.isbn() != item2.isbn())
    throw runtime_error("Data must refer to same ISBN");
// 如果程序执行到了这里, 表示两个 ISBN 是相同的
cout << item1 + item2 << endl;
```

在这段代码中, 如果 ISBN 不一样就抛出一个异常, 该异常是类型 `runtime_error` 的对象。抛出异常将终止当前的函数, 并把控制权转移给能处理该异常的代码。

类型 `runtime_error` 是标准库异常类型的一种, 定义在 `stdexcept` 头文件中。关于标准库异常类型更多的知识将在 5.6.3 节 (第 176 页) 介绍。我们必须初始化 `runtime_error` 的对象, 方式是给它提供一个 `string` 对象或者一个 C 风格的字符串 (参见 3.5.4 节, 第 109 页), 这个字符串中有一些关于异常的辅助信息。

5.6.2 try 语句块

try 语句块的通用语法形式是

```
try {
    program-statements
} catch (exception-declaration) {
    handler-statements
} catch (exception-declaration) {
    handler-statements
} // ...
```

try 语句块的一开始是关键字 `try`, 随后紧跟着一个块, 这个块就像大多数时候那样是花括号括起来的语句序列。

195

跟在 `try` 块之后的是一个或多个 `catch` 子句。 `catch` 子句包括三部分: 关键字 `catch`、括号内一个 (可能未命名的) 对象的声明 (称作异常声明, `exception declaration`) 以及一个块。当选中了某个 `catch` 子句处理异常之后, 执行与之对应的块。 `catch` 一旦完成, 程序跳转到 `try` 语句块最后一个 `catch` 子句之后的那条语句继续执行。

try 语句块中的 *program-statements* 组成程序的正常逻辑, 像其他任何块一样, *program-statements* 可以有包括声明在内的任意 C++ 语句。一如往常, try 语句块内声明的变量在块外部无法访问, 特别是在 `catch` 子句内也无法访问。

编写处理代码

在之前的例子里, 我们使用了一个 `throw` 表达式以避免把两个代表不同书籍的 `Sales_item` 相加。我们假设执行 `Sales_item` 对象加法的代码是与用户交互的代码分离开来的。其中与用户交互的代码负责处理发生的异常, 它的形式可能如下所示:

```
while (cin >> item1 >> item2) {
    try {
        // 执行添加两个 Sales_item 对象的代码
        // 如果添加失败, 代码抛出一个 runtime_error 异常
    } catch (runtime_error err) {
        // 提醒用户两个 ISBN 必须一致, 询问是否重新输入
        cout << err.what()
             << "\nTry Again? Enter y or n" << endl;
        char c;
        cin >> c;
```

```
        if (!cin || c == 'n')
            break; // 跳出 while 循环
    }
}
```

程序本来要执行的任务出现在 try 语句块中，这是因为这段代码可能会抛出一个 runtime_error 类型的异常。

try 语句块对应一个 catch 子句，该子句负责处理类型为 runtime_error 的异常。如果 try 语句块的代码抛出了 runtime_error 异常，接下来执行 catch 块内的语句。在我们书写的 catch 子句中，输出一段提示信息要求用户指定程序是否继续。如果用户输入 'n'，执行 break 语句并退出 while 循环；否则，直接执行 while 循环的右侧花括号，意味着程序控制权跳回到 while 条件部分准备下一次迭代。

给用户的提示信息中输出了 err.what() 的返回值。我们知道 err 的类型是 runtime_error，因此能推断 what 是 runtime_error 类的一个成员函数（参见 1.5.2 节，第 20 页）。每个标准库异常类都定义了名为 what 的成员函数，这些函数没有参数，返回值是 C 风格字符串（即 const char*）。其中，runtime_error 的 what 成员返回的是初始化一个具体对象时所用的 string 对象的副本。如果上一节编写的代码抛出异常，则本节的 catch 子句输出

< 196

```
Data must refer to same ISBN
Try Again? Enter y or n
```

函数在寻找处理代码的过程中退出

在复杂系统中，程序在遇到抛出异常的代码前，其执行路径可能已经经过了多个 try 语句块。例如，一个 try 语句块可能调用了包含另一个 try 语句块的函数，新的 try 语句块可能调用了包含又一个 try 语句块的新函数，以此类推。

寻找处理代码的过程与函数调用链刚好相反。当异常被抛出时，首先搜索抛出该异常的函数。如果没找到匹配的 catch 子句，终止该函数，并在调用该函数的函数中继续寻找。如果还是没有找到匹配的 catch 子句，这个新的函数也被终止，继续搜索调用它的函数。以此类推，沿着程序的执行路径逐层回退，直到找到适当类型的 catch 子句为止。

如果最终还是没能找到任何匹配的 catch 子句，程序转到名为 terminate 的标准库函数。该函数的行为与系统有关，一般情况下，执行该函数将导致程序非正常退出。

对于那些没有任何 try 语句块定义的异常，也按照类似的方式处理：毕竟，没有 try 语句块也就意味着没有匹配的 catch 子句。如果一段程序没有 try 语句块且发生了异常，系统会调用 terminate 函数并终止当前程序的执行。

提示：编写异常安全的代码非常困难

要好好理解这句话：异常中断了程序的正常流程。异常发生时，调用者请求的一部分计算可能已经完成了，另一部分则尚未完成。通常情况下，略过部分程序意味着某些对象处理到一半就戛然而止，从而导致对象处于无效或未完成的状态，或者资源没有正常释放，等等。那些在异常发生期间正确执行了“清理”工作的程序被称作异常安全（exception safe）的代码。然而经验表明，编写异常安全的代码非常困难，这部分知识也（远远）超出了本书的范围。

对于一些程序来说，当异常发生时只是简单地终止程序。此时，我们不怎么需要担

心异常安全的问题。

但是对于那些确实要处理异常并继续执行的程序，就要加倍注意了。我们必须时刻清楚异常何时发生，异常发生后程序应如何确保对象有效、资源无泄漏、程序处于合理状态，等等。

我们会在本书中介绍一些比较常规的提升异常安全性的技术。但是读者需要注意，如果你的程序要求非常鲁棒的异常处理，那么仅有我们介绍的这些技术恐怕还是不够的。

197

5.6.3 标准异常

C++标准库定义了一组类，用于报告标准库函数遇到的问题。这些异常类也可以在用户编写的程序中使用，它们分别定义在 4 个头文件中：

- exception 头文件定义了最通用的异常类 exception。它只报告异常的发生，不提供任何额外信息。
- stdexcept 头文件定义了几种常用的异常类，详细信息在表 5.1 中列出。
- new 头文件定义了 bad_alloc 异常类型，这种类型将在 12.1.2 节（第 407 页）详细介绍。
- type_info 头文件定义了 bad_cast 异常类型，这种类型将在 19.2 节（第 731 页）详细介绍。

表 5.1: <stdexcept>定义的异常类	
exception	最常见的问题
runtime_error	只有在运行时才能检测出的问题
range_error	运行时错误：生成的结果超出了有意义的值域范围
overflow_error	运行时错误：计算上溢
underflow_error	运行时错误：计算下溢
logic_error	程序逻辑错误
domain_error	逻辑错误：参数对应的结果值不存在
invalid_argument	逻辑错误：无效参数
length_error	逻辑错误：试图创建一个超出该类型最大长度的对象
out_of_range	逻辑错误：使用一个超出有效范围的值

标准库异常类只定义了几种运算，包括创建或拷贝异常类型的对象，以及为异常类型的对象赋值。

我们只能以默认初始化（参见 2.2.1 节，第 40 页）的方式初始化 exception、bad_alloc 和 bad_cast 对象，不允许为这些对象提供初始值。

其他异常类型的行为则恰好相反：应该使用 string 对象或者 C 风格字符串初始化这些类型的对象，但是不允许使用默认初始化的方式。当创建此类对象时，必须提供初始值，该初始值含有错误相关的信息。

异常类型只定义了一个名为 what 的成员函数，该函数没有任何参数，返回值是一个指向 C 风格字符串（参见 3.5.4 节，第 109 页）的 const char*。该字符串的目的是提供关于异常的一些文本信息。

`what` 函数返回的 C 风格字符串的内容与异常对象的类型有关。如果异常类型有一个字符串初始值，则 `what` 返回该字符串。对于其他无初始值的异常类型来说，`what` 返回的内容由编译器决定。

< 198

5.6.3 节练习

练习 5.23: 编写一段程序，从标准输入读取两个整数，输出第一个数除以第二个数的结果。

练习 5.24: 修改你的程序，使得当第二个数是 0 时抛出异常。先不要设定 `catch` 子句，运行程序并真的为除数输入 0，看看会发生什么？

练习 5.25: 修改上一题的程序，使用 `try` 语句块去捕获异常。`catch` 子句应该为用户输出一条提示信息，询问其是否输入新数并重新执行 `try` 语句块的内容。

199 小结

C++语言仅提供了有限的语句类型，它们中的大多数会影响程序的控制流程：

- while、for 和 do while 语句，执行迭代操作。
- if 和 switch 语句，提供条件分支结构。
- continue 语句，终止循环的当前一次迭代。
- break 语句，退出循环或者 switch 语句。
- goto 语句，将控制权转移到一条带标签的语句。
- try 和 catch，将一段可能抛出异常的语句序列括在花括号里构成 try 语句块。catch 子句负责处理代码抛出的异常。
- throw 表达式语句，存在于代码块中，将控制权转移到相关的 catch 子句。
- return 语句，终止函数的执行。我们将在第 6 章介绍 return 语句。

除此之外还有表达式语句和声明语句。表达式语句用于求解表达式，关于变量的声明和定义在第 2 章已经介绍过了。

术语表

块 (block) 包围在花括号内的由 0 条或多条语句组成的序列。块也是一条语句，所以只要是能使用语句的地方，就可以使用块。

break 语句 (break statement) 终止离它最近的循环或 switch 语句。控制权转移到循环或 switch 之后的第一条语句。

case 标签 (case label) 在 switch 语句中紧跟在 case 关键字之后的常量表达式 (参见 2.4.4 节, 第 58 页)。在同一个 switch 语句中任意两个 case 标签的值不能相同。

catch 子句 (catch clause) 由三部分组成：catch 关键字、括号里的异常声明以及一个语句块。catch 子句的代码负责处理在异常声明中定义的异常。

复合语句 (compound statement) 和块是同义词。

continue 语句 (continue statement) 终止离它最近的循环的当前迭代。控制权转移到 while 或 do while 语句的条件部分、或者范围 for 循环的下次迭代、或者传统 for 循环头部的表达式。

悬垂 else (dangling else) 是一个俗语，指的是如何处理嵌套 if 语句中 if 分支多于 else 分支的情况。C++语言规定，else 应该与前一个未匹配的 if 匹配在一起。使用花括号可以把位于内层的 if 语句隐藏起来，这样程序员就能更好地控制 else 该与哪个 if 匹配。

default 标签 (default label) 是一种特殊的 case 标签，当 switch 表达式的值与所有 case 标签都无法匹配时，程序执行 default 标签下的内容。

do while 语句 (do while statement) 与 while 语句类似，区别是 do while 语句先执行循环体，再判断条件。循环体代码至少会执行一次。

异常类 (exception class) 是标准库定义的一组类，用于表示程序发生的错误。表 5.1 (第 176 页) 列出了不同用途的异常类。

异常声明 (exception declaration) 位于 catch 子句中的声明，指定了该 catch 子句能处理的异常类型。

异常处理代码 (exception handler) 程序某处引发异常后，用于处理该异常的另一